Learning Inverse Kinematics by Imitating a Numerical Solver: Analysis and Improvements on Two 3-DOF Manipulators

Durgesh S. Reddiyar
Ira A. Fulton Schools of Engineering
Arizona State University, Tempe, AZ, USA
dreddiya@asu.edu

***Abstract*—To move a robot arm's hand to a target, we need its inverse kinematics (IK): the joint values that place the hand there. A popular shortcut is to train a neural network for IK, using an existing numerical IK solver to create the training examples. This report studies such a system, built with the MATLAB Robotics Toolbox and Keras for two simple three-joint arms: an all-rotary (RRR) arm and a cylindrical (RPP) arm. Measured by how far the hand actually lands from the target, the baseline network misses by a median of about 4% of the arm's reach, and by more than half the reach for the worst 5% of targets. Keras's default "accuracy" metric gives this network 74% and cannot reveal the problem, because it only checks which of the three joint values is largest. We trace most of the error to the training data. The solver often returns different, equally valid joint solutions for neighbouring targets, so the network is asked to learn an inconsistent mapping. The solver's failures come simply from targets outside the arm's reach. And one data generator confuses a position coordinate with an angle. Fixing the data makes the network $43\times$ more accurate at the median and about $200\times$ more accurate on the hardest targets. The fixes are to always use the same solution branch, to compute exact labels, and to predict the sine and cosine of each angle. The improved network also makes a good starting guess for the numerical solver: it needs $3.5\times$ fewer iterations and always returns the intended solution. We release the code and summarise the lessons as a short checklist.**

***Index Terms*—Inverse kinematics, neural networks, robot manipulators, Levenberg–Marquardt, reproducibility, evaluation metrics.**

I. Introduction

Inverse kinematics asks for joint variables $\mathbf{q}$ that place a manipulator's end effector at a desired pose. For most serial arms, the forward map $\mathbf{p} = \mathrm{FK}(\mathbf{q})$ is a closed-form composition of homogeneous transforms [1], [2]. Its inverse, however, is a set-valued map: a target can have no solution, a finite number of them, or a continuum. Classical treatment splits into analytic solutions for special geometries [3]–[5] and iterative Jacobian methods for the general case [6]–[10].

Learning the inverse map with a neural network is a long-standing idea [11]–[16]. A trained network gives an answer in a single, constant-time forward pass. Universal-approximation results [17] make it tempting to treat IK as plain regression. It has been known since at least Jordan and Rumelhart [12] that this view is incomplete. When the target relation is one-to-many, a least-squares regressor averages solutions that may not themselves be solutions. Recent work addresses multiplicity explicitly through structured prediction [15], mixture or generative models [18]–[20], or by partitioning the data [16].

Despite this literature, a very common first implementation of "IK with a neural network" consists of four steps: (1) sample random Cartesian targets; (2) label them by running an off-the-shelf numerical solver; (3) fit an MLP with a stock deep-learning API; and (4) report the API's default metric together with a few qualitative comparisons. This report presents such a pipeline, developed at Arizona State University for two textbook 3-DOF arms [2] using Peter Corke's Robotics Toolbox for MATLAB [21], [22] and Keras/TensorFlow [23], [24].

We describe the pipeline in full, evaluate it in task space, identify what limits its accuracy, and show how to fix it. The pitfalls we find, an uninformative default metric and inconsistent training labels, are common well beyond this project [25], [26].

a) Contributions:

- **A complete description of the baseline pipeline** (Section IV). This includes the network architecture, loss, optimiser, and training split, all taken directly from the saved models and code.
- **An analysis of the data, the metric, and the evaluation** (Section V). It identifies the causes of the baseline's error summarised in the abstract.
- **A task-space evaluation** of all seven trained models (Section VI), including generalisation gaps and failure geometry.
- **A controlled study** (Section VII) that keeps the inputs fixed and isolates the effects of label consistency, angle wrapping, loss, output representation, and data scale. It also covers a hybrid learned-warm-start solver and latency measurements.
- **A practical checklist** (Section VIII) and a self-contained release of the analysis code.

II. Background and Related Work

a) Numerical IK: Iterative methods linearise FK around the current estimate and solve for a joint update. The undamped pseudo-inverse step is ill-conditioned near singular configurations. Damped least squares [6], [7] and its adaptive

form, the Levenberg–Marquardt (LM) method [9], [27], [28], trade step accuracy for robustness. The Robotics Toolbox (RTB) `ikine` used here implements an LM scheme with a multiplicatively adapted damping factor [22]. Production libraries such as TRAC-IK [10] add random restarts and joint-limit handling. Two properties of all local methods matter for this paper. First, the solution they return depends on the initial guess, i.e. on which basin of attraction the guess falls into. Second, joint angles are accumulated without wrapping, so a returned angle is only defined up to multiples of 2π.

b) Analytic IK: Arms whose last three axes intersect, and the two 3-DOF arms studied here, admit closed-form solutions [2], [3]. IKFast generates such solvers automatically [5]. When an analytic solution exists, a learned model is redundant as a *solver*. Such arms remain valuable as *testbeds*, however, because exact ground truth and the full solution set are available. This is how we use them in Section VII.

c) Learned IK: Early neural IK work [11] treated the inverse as direct regression. Jordan and Rumelhart [12] showed that, for non-convex inverse sets, direct inverse modelling converges to averages of valid solutions that are not solutions themselves. Their *distal teacher* instead trains through a forward model. Later work localised the problem with local linear models [13], modular experts [14], structured output learning [15], heuristic splitting of the dataset [16], or explicit conditional densities via mixture density networks [18], invertible networks [19], and normalising flows [20]. Output parameterisation also matters: angle outputs are discontinuous at the wrap point, and continuous embeddings such as $(\cos\theta, \sin\theta)$ are preferable [29].

d) Reproducibility: Reproducibility programmes [25] and studies of data leakage in ML-based science [26] document how evaluation errors, especially train/test contamination and inappropriate metrics, can distort reported performance. IK is a convenient setting in which to study such effects, because exact ground truth makes their size measurable.

III. Problem Setup

Both manipulators are taken verbatim from the project code and use standard Denavit–Hartenberg (DH) conventions [1]. We use the convention of RTB's `Link([theta d a alpha sigma])`. Lengths are in the toolbox's unitless "length units" (LU).

a) Anthropomorphic RRR arm: The DH parameters are $(a_i, \alpha_i, d_i) = (0, \pi/2, 0)$, $(a_2, 0, 0)$, and $(a_3, 0, 0)$, with $a_2 = a_3 = 10$ LU. The forward kinematics of the wrist point are

$$\mathbf{p} = \begin{bmatrix} c_1 r \\ s_1 r \\ a_2 s_2 + a_3 s_{23} \end{bmatrix}, \qquad r = a_2 c_2 + a_3 c_{23}, \tag{1}$$

where $c_i = \cos\theta_i$, $s_i = \sin\theta_i$, and $c_{23} = \cos(\theta_2 + \theta_3)$. The workspace is the closed ball $\|\mathbf{p}\| \le a_2 + a_3 = 20$. Away from singularities, every interior target has four solutions, indexed by a shoulder sign $\sigma_s \in \{\pm 1\}$ and an elbow sign $\sigma_e \in \{\pm 1\}$:

$$\begin{aligned} \theta_1 &= \operatorname{atan2}(p_y, p_x) + \tfrac{1-\sigma_s}{2}\pi, \\ c_3 &= \frac{\|\mathbf{p}\|^2 - a_2^2 - a_3^2}{2 a_2 a_3}, \\ \theta_3 &= \operatorname{atan2}\big(\sigma_e \sqrt{1 - c_3^2},\, c_3\big), \\ \theta_2 &= \operatorname{atan2}(p_z,\, \sigma_s \rho) - \operatorname{atan2}(a_3 s_3,\, a_2 + a_3 c_3), \end{aligned} \tag{2}$$

with $\rho = \sqrt{p_x^2 + p_y^2}$. The singular set consists of three parts:

- the z-axis, $\rho = 0$ (shoulder singularity, where θ_1 is undefined);
- the boundary sphere, $\|\mathbf{p}\| = 20$ (outstretched elbow, $\theta_3 = 0$);
- the origin (folded elbow, $\theta_3 = \pi$), which is reachable because $a_2 = a_3$.

b) Cylindrical RPP arm: The DH parameters are $(\theta_1, d_1{=}5, 0, 0)$ (revolute), $(0, d_2, 0, -\pi/2)$ (prismatic), and $(0, d_3, 0, 0)$ (prismatic). Hence

$$\begin{aligned} \mathbf{p} &= \big[-s_1 d_3,\; c_1 d_3,\; d_1 + d_2\big]^\top, \\ \theta_1 &= \operatorname{atan2}(-p_x,\, p_y), \quad d_2 = p_z - d_1, \quad d_3 = \rho. \end{aligned} \tag{3}$$

With $d_3 \ge 0$, the solution is unique (up to 2π in θ_1) except on the z-axis.

c) Metrics: The quantity that matters for a positioning task is the *task-space error* $e(\mathbf{p}) = \|\mathrm{FK}(\hat{\mathbf{q}}(\mathbf{p})) - \mathbf{p}\|$. Unlike joint-space error, it does not depend on which of several valid solutions a model returns, or on how angles are wrapped. We report its median, mean, and tail percentiles, and joint errors only after wrapping differences to $(-\pi, \pi]$.

IV. The Original Pipeline, As Implemented

Figure 1 summarises the system. The model and training details below are taken directly from the HDF5 model files, which store the full Keras model and training configuration (Keras 2.3.1, TensorFlow backend), and from `train.py`.

a) Data generation (`gen.m`, `gen_rpp.m`): For the RRR arm, 10^4 targets are drawn uniformly from the cube $[-15, 15]^3$. Each target is passed, as a pure translation, to `SerialLink.ikine` with a position-only mask (`'mask',[1 1 1 0 0 0]`), the initial guess $\mathbf{q}_0 = \mathbf{0}$, and `'ilimit',1000`. Targets for which the solver returns an empty result are dropped. For the RPP arm, the script draws $x \sim \mathcal{U}[\mathrm{deg2rad}(0), \mathrm{deg2rad}(360)]$ and $y, z \sim \mathcal{U}[5, 200]$. We return to this choice in Section V-C. An earlier dataset in the repository (`train_rpp.csv`) used $x \sim \mathcal{U}[0, 360]$ instead.

b) Model: All production models share the architecture

$$3\text{–}32\text{–}64\text{–}128\text{–}256\text{–}256_{\sigma}\text{–}256\text{–}128\text{–}128\text{–}64\text{–}3,$$

with ReLU activations, one sigmoid layer (marked σ), a linear output, and 233,027 parameters. Training minimises the mean absolute error (MAE) with Adam [30] ($\eta = 10^{-3}$, default moments) at batch size 768, with `metrics=['accuracy']`. Inputs and outputs are not normalised. The data are split

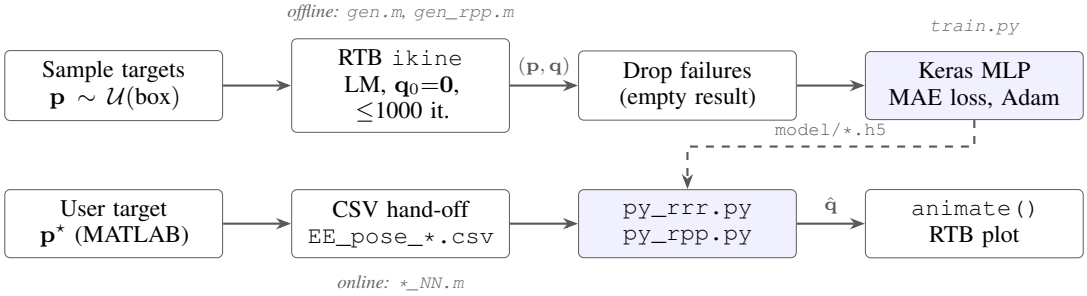


Fig. 1. The original solver-supervised IK pipeline. Offline, random Cartesian targets are labelled by the Robotics Toolbox LM solver and an MLP is fitted to the successful pairs. Online, MATLAB passes a target to the network through CSV files and displays the arm with the Robotics Toolbox.

80:20 with `train_test_split(random_state=1)`. The train/test gap in Table III matches this split to a recorded training run. The RRR model was checkpointed after 2k (smaller architecture), 10k, 15k, and 20k epochs; the 20k-epoch model ships with the demo.

c) Deployment: MATLAB front-ends (`*_NN.m`, `*_toolbox.m`) write a target to CSV, call the Python inference scripts, and display the arm with the toolbox's `animate()`. They serve visualisation only. All results in this report are computed numerically from the trained models and datasets.

d) Artifacts: The repository contains 7 Keras models, 5 datasets, 2 generators, 4 MATLAB front-ends, and 3 Python scripts (Table I). All analyses use them unmodified. Since MATLAB was not available for this analysis, we re-implemented the kinematics and the RTB LM solver in Python (Section A) and loaded the Keras weights into a NumPy forward pass. The NumPy pass is checked against the stored training metrics: for example, it reproduces the stored Keras test "accuracy" of 74.1% (Section V-D).

V. Analysis of the Baseline Pipeline

A. Why does the solver fail? Workspace, not singularities

Of the 10^4 RRR targets, 819 (8.19%) produced no solution. Two causes are plausible a priori: the 1000-iteration limit and singular target poses.

Finding 1. *Unreachable targets explain the failure rate.* Every target with $\|\mathbf{p}\| > 20$ is outside the workspace, and the sampling cube $[-15, 15]^3$ has corners at distance $15\sqrt{3} \approx 26$. By Monte Carlo (2×10^6 samples), 7.86% of the cube is unreachable (Figure 2a). Running our LM replica on a fresh draw of 10^4 targets from the same cube fails on 7.84%. All 784 failures are unreachable targets, and not a single reachable target fails. Successful solves take a median of 14 iterations (95th percentile 28, maximum 51), so the 1000-iteration cap is never close to binding. Consistently, the largest target norm in the original dataset is 19.9999. The solver labels themselves are accurate: $\|\mathrm{FK}(\mathbf{q}) - \mathbf{p}\|$ has a median of 4.3×10^{-4} and a maximum of 5.3×10^{-3}. Singular configurations do degrade the conditioning of the LM step, but the adaptive damping handles them. The retained data accordingly come within 1.3×10^{-4} LU of the boundary sphere (51 targets with $\|\mathbf{p}\| > 19.9$), within 0.14 LU of the z-axis, and within 0.51 LU of the origin, which is as close as 10^4 uniform samples get. Dropping failed samples therefore removes unreachable targets, not singular ones.

B. The labels are not a well-posed regression target

Finding 2. *The solver's branch choice is spatially incoherent.* Classifying each label by Equation (2), the shoulder is "front" (σ_s=+1) for 81.4% of samples, and the elbow splits 49.7% / 50.3%. Branch membership is not a smooth function of $\mathbf{p}$. For a held-out target, its nearest training target (median distance 0.84 LU, i.e. 4% of the reach) lies on the same full branch only 66.0% of the time: 74.5% for the elbow alone and 86.2% for the shoulder alone (Figure 2b). Starting every solve from $\mathbf{q}_0 = \mathbf{0}$ yields a fixed but fragmented partition of the workspace into basins of attraction. A regressor must either memorise this partition, at the cost of poor generalisation, or average across it.

Finding 3. *Labels are unwrapped.* 117 samples (1.27%) contain a joint value outside $(-\pi, \pi]$, up to $q_1 = -71.9$ and $q_3 = 63.8$ rad (Figure 2c). For MAE or MSE training, these points sit 10–70 radians away from geometrically identical labels.

Our LM replica reproduces the branch statistics closely: shoulder front 80.7% vs. 81.4% in the original data, elbow-up 50.6% vs. 49.7%. However, it unwraps angles far more often (66% of solutions). We attribute this to step-control details of the specific toolbox release, which we cannot run. Our conclusions use the original labels wherever possible and use the replica only for failure and convergence statistics, which it matches.

C. The RPP sampler draws a Cartesian coordinate from an angle range

Finding 4. In `gen_rpp.m`, the first input coordinate is sampled between `deg2rad(0)` and `deg2rad(360)`, so $x \in [0, 2\pi]$ while $y, z \in [5, 200]$. The earlier `train_rpp.csv` used $x \in [0, 360]$. This looks like a conflation of the target's x coordinate with the base angle θ_1. The effect on coverage is drastic: training targets lie in a slab 6.3 LU thick, so the base angle spans only $\theta_1 \in [-49.4°, 0°]$, 13.7% of the full rotation (Figure 3b). The resulting `model8000_rpp_rad` is

TABLE I
INVENTORY OF THE PROJECT ARTIFACTS.

Artifact	Content (as found)
`gen.m`	RRR: 10^4 targets $\sim \mathcal{U}[-15,15]^3$, `ikine`($\mathbf{q}_0$=$\mathbf{0}$, mask $[1\,1\,1\,0\,0\,0]$, `ilimit` 1000) $\rightarrow$ `train_3link.csv` (9,181 rows)
`gen_rpp.m`	RPP: $x \sim \mathcal{U}[0,2\pi]$, $y,z \sim \mathcal{U}[5,200] \rightarrow$ `train_rpp_rad.csv` (10,000 rows); earlier `train_rpp.csv` with $x \sim \mathcal{U}[0,360]$ (7,120 rows)
`gen.py`	Analytic RPP labels, $x \leq 0$ variant $\rightarrow$ `train10000test.csv`; `train8000test.csv` (8,000 rows, $x \in [0,200]$)
`train.py`	Keras MLP (Section IV), MAE, Adam, batch 768, `train_test_split(0.2, random_state=1)`
`model*_rrr.h5`	2k (7-layer, sigmoid output, 86,915 params), 10k, 15k, 20k epochs (233,027 params)
`model8000_rpp*.h5`	RPP models for the two RPP datasets (233,027 params)
`model2000.h5`	RPP model on `train8000test.csv`; test median error 12.2 LU
`*_toolbox.m`, `*_NN.m`	MATLAB front-ends: CSV hand-off, `animate()` loops
`py_rrr.py`, `py_rpp.py`	Load a model, predict for the example target $[1,4,9]$, write CSV

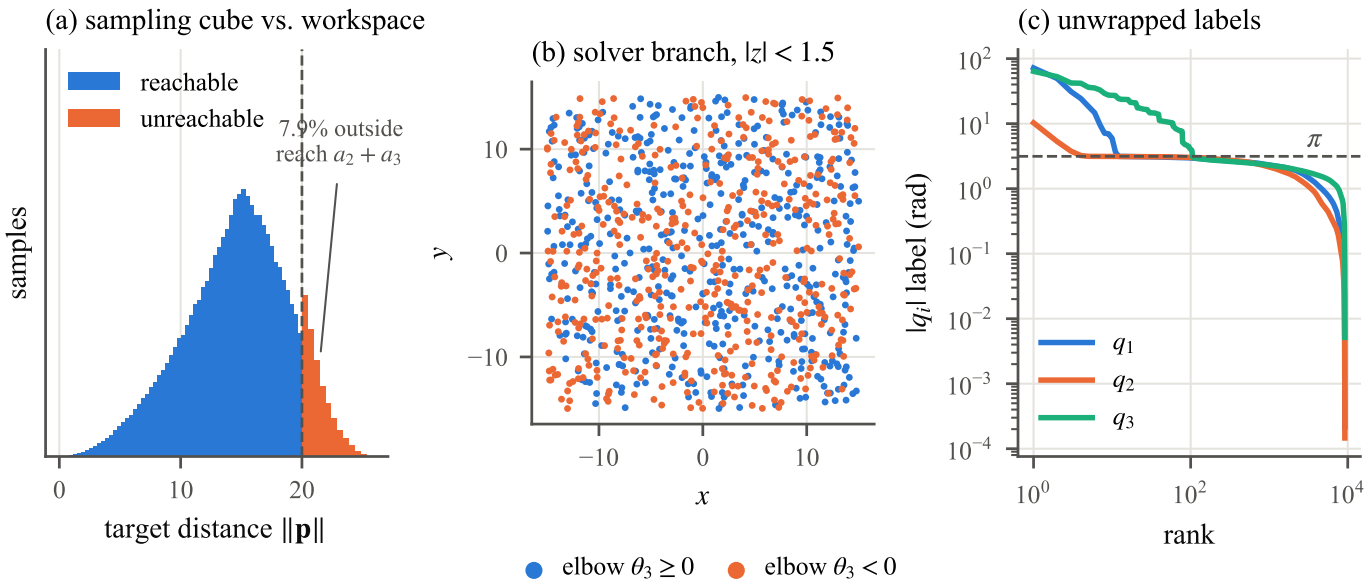


Fig. 2. Dataset pathologies (RRR). **(a)** Distribution of $\|\mathbf{p}\|$ for targets drawn uniformly from $[-15,15]^3$. Everything beyond the reach $a_2{+}a_3{=}20$ (7.9%) is unsolvable, which explains the 8.2% failure rate. **(b)** Elbow branch of the solver's label for dataset targets in the slab $|z| < 1.5$. Neighbouring targets frequently receive solutions on different branches. **(c)** Sorted absolute joint labels on log axes. A tail of labels exceeds π by up to 23$\times$ because the solver does not wrap angles.

accurate in-distribution (median task error 2.1 LU on a $\sim$200-LU workspace). But it degrades linearly as soon as x leaves the slab (Figure 3a). On the input suggested in the RPP scripts, $[200,200,100]$, it returns $\hat{\mathbf{q}} = [0.087, 46.1, 175.0]$ against the exact $[-0.785, 95.0, 282.8]$, an error of 222 LU. For the earlier $x \in [0,360]$ data, our replica fails on 29.7% of targets, against 28.8% in the original file. All RPP labels are consistent with Equation (3) to within 0.13 LU.

D. What Keras's default accuracy measures

Finding 5. With `metrics=['accuracy']`, a three-unit output, and a non-cross-entropy loss, Keras resolves "accuracy" to *categorical* accuracy,

$$\text{acc} = \frac{1}{N}\sum_n \mathbb{1}\big[\arg\max_j \hat{q}_{n,j} = \arg\max_j q_{n,j}\big].$$

This only checks whether the *largest of the three joint values* is in the same position. On its test split, `model20000_rrr` scores 74.1%; on the training split, 97.3%. The metric is uninformative in both directions (Table III). The RPP model from the $x \in [0,360]$ data scores 91.4% while its median position error is 148 LU, because its argmax is almost always d_3 (majority-class rate 81.0%). Conversely, it could not distinguish a sub-millimetre solver from one that is off by 10° in every joint.

E. Worked example

For $\mathbf{p} = [1,4,9]^\top$ ($\|\mathbf{p}\| = 9.90$), the four exact solutions are listed in Table II. Any converged position-only solver must return one of them modulo 2π. Our LM replica, started at $\mathbf{0}$, returns the $(+,+)$ branch $[75.96^\circ, 5.05^\circ, 120.66^\circ]$ after 13 iterations, modulo 2π, but in unwrapped form. The base-

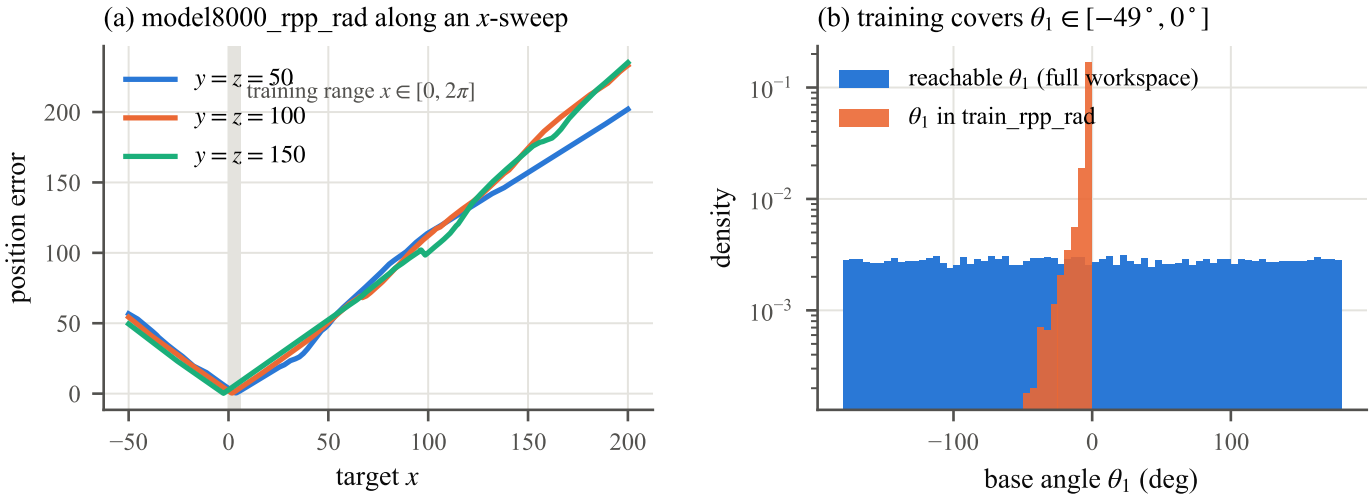


Fig. 3. RPP arm. **(a)** Task-space error of the original RPP network along x-sweeps at three heights. The shaded band is the training range of x, created by sampling x from an angle interval. **(b)** Base angles covered by the training set (orange) versus the full reachable range (blue, log scale).

line `model20000_rrr` predicts $[78.77°, 4.38°, 121.21°]$, on the same branch. It deviates from the exact solution by $(2.81°, 0.67°, 0.55°)$, and its end effector lands 0.23 LU (2.3% of $\|\mathbf{p}\|$) from the target. This target is representative of the median case. The tail of the error distribution (Section VI) is dominated by targets on which the network switches branch.

VI. Task-Space Evaluation of the Baseline Models

Table III evaluates all original models on their own (recovered) splits. Three points stand out.

a) Overfitting to the branch partition: On its training split, `model20000_rrr` has a wrapped joint MAE of 4–5° and a median task error of 0.50 LU. On held-out targets, the joint MAE rises to 27–33°, and the mean task error more than doubles (0.85 to 2.19 LU). The test median is 0.78 LU (3.9% of the reach), and the tail is heavy: the 95th percentile is 10.9 LU, more than half the reach, and only 0.5% of test targets are within 0.1 LU (Figure 4a). The joint-space errors far exceed what the task-space errors would suggest, which is a signature of branch switching. The network lands on the label's branch for 68.8% of test targets, with a median error of 0.60 LU; elsewhere the median error is 1.51 LU and the 95th percentile is 19.7 LU. The 26 test targets with unwrapped labels have a median error of 6.6 LU. Training longer (10k→20k epochs) lowers the held-out median from 0.96 to 0.78 LU and the 95th percentile from 14.0 to 10.9 LU, while the train/test gap widens.

b) Geometry of the errors: The median error is highest for targets near the origin (Figure 4b), where the folded-elbow singularity makes θ_2 and θ_3 ill-defined. It is also mildly elevated near the z-axis, where θ_1 is undefined. A constant predictor (the mean training label) has a median error of 23.2 LU, so the network clearly learns most of the map. The question is whether the residual matters for the application, and Table III shows that it usually does.

c) The RPP models: `model8000_rpp_rad` is the best-behaved model in the repository in-distribution: its θ_1 error is 1.5° and its median position error is 2.1 LU. This is expected, because the RPP inverse is single-valued and smooth on its training slab. Its failure is entirely one of coverage (Section V-C). `model8000_rpp` (data with $x \in [0, 360]$) never learned a useful map: its median error is 148 LU, despite a Keras accuracy of 91.4%.

VII. Controlled Study: What Makes Solver-Supervised IK Work?

The analysis identifies several suspects: inconsistent branches, unwrapped angles, the loss, and the output parameterisation. To separate their effects, we train identical networks on *identical inputs*, namely the 7,344 training targets of the original RRR split, and vary one factor at a time. The RRR arm admits an exact, enumerable solution set (Equation (2)), so the label for each regime can be generated exactly.

a) Setup: The five label regimes are:

- the solver's labels as generated;
- the same labels wrapped to $(-\pi, \pi]$;
- exact closed-form labels with a branch drawn uniformly at random per sample, i.e. maximal multiplicity;
- exact labels on one consistent branch, $(\sigma_s, \sigma_e) = (+, +)$, with angle outputs;
- the same consistent labels with a $(\cos q_i, \sin q_i)$ output head, decoded with atan2.

Each regime is trained with an L1 (MAE, as in the original) and an L2 (MSE) loss, three seeds each. The network is an MLP with three hidden layers of 256 ReLU units and fixed input scaling $\mathbf{p}/15$. Training uses Adam [30] at $\eta = 2 \times 10^{-3}$ with cosine decay, 40k steps, and batch size 512, in PyTorch [31]. We evaluate on 20,000 fresh targets drawn uniformly from the reachable part of the original sampling cube. The original test split gives the same ranking.

TABLE II
SOLUTIONS FOR THE EXAMPLE TARGET $\mathbf{p} = [1, 4, 9]^\top$.

Source	$(\theta_1, \theta_2, \theta_3)$ [deg]	$\mathrm{FK}(\mathbf{q})$	$\|\mathrm{FK}(\mathbf{q}) - \mathbf{p}\|$
Exact, $(\sigma_s, \sigma_e) = (+, +)$	$(75.96,\ 5.05,\ 120.66)$	$(1, 4, 9)$	0
Exact, $(+, -)$	$(75.96,\ 125.72,\ -120.66)$	$(1, 4, 9)$	0
Exact, $(-, +)$	$(-104.04,\ 54.28,\ 120.66)$	$(1, 4, 9)$	0
Exact, $(-, -)$	$(-104.04,\ 174.95,\ -120.66)$	$(1, 4, 9)$	0
LM replica from $\mathbf{0}$ (13 it.)	$(75.96,\ -354.95,\ 840.66)$	$(1, 4, 9)$	$< 10^{-10}$
`model20000_rrr`	$(78.77,\ 4.38,\ 121.21)$	$(0.81, 4.07, 8.90)$	0.23

TABLE III
ORIGINAL MODELS ON THEIR RECOVERED TRAIN/TEST SPLITS. "KERAS ACC." IS KERAS'S DEFAULT '`ACCURACY`' METRIC (SECTION V-D). "MAJ." IS THE CATEGORICAL ACCURACY OF ALWAYS PREDICTING THE MOST FREQUENT ARGMAX INDEX ON THE TEST SPLIT. JOINT MAE USES WRAPPED DIFFERENCES. TASK-SPACE ERRORS ARE IN LU (RRR REACH 20; RPP WORKSPACE $\sim$200). THE LAST COLUMN IS THE SHARE OF TARGETS WITH ERROR $<$ 1 LU.

Model	Arm	Split	Keras acc.	Maj.	Joint MAE [deg]	Task-space error [LU]			< 1 LU
			[%]	[%]	(q_1, q_2, q_3)	median	mean	p95	[%]
`model2000_rrr`	RRR	test	67.7	37.3	53, 46, 74	17.96	18.26	31.89	1.1
`model10000_rrr`	RRR	test	74.0	37.3	27, 34, 33	0.96	2.67	13.98	51.9
`model15000_rrr`	RRR	test	74.4	37.3	26, 34, 33	0.79	2.30	11.72	60.7
`model20000_rrr`	RRR	test	74.1	37.3	27, 33, 32	0.78	2.19	10.92	61.6
`model20000_rrr`	RRR	train	97.3	37.3	4, 4, 5	0.50	0.85	2.14	82.5
`model8000_rpp_rad`	RPP	test	99.9	51.7	1.5 (q_1)	2.11	2.87	7.70	–
`model8000_rpp`	RPP	test	91.4	81.0	27.4 (q_1)	148.16	155.55	352.60	–

TABLE IV
CONTROLLED STUDY ON THE RRR ARM: IDENTICAL INPUTS (7,344 TARGETS), ARCHITECTURE, AND OPTIMISER; ONLY THE LABELS, LOSS, AND OUTPUT HEAD CHANGE. THE TABLE REPORTS TASK-SPACE ERROR ON 20K FRESH REACHABLE TARGETS (MEAN $\pm$ S.D. OVER 3 SEEDS; LU, REACH $= 20$). THE "$<$0.1" COLUMNS GIVE THE SHARE OF TARGETS WITH ERROR BELOW 0.1 LU, IN %.

	L1 loss			L2 loss		
Labels / output	median	p95	$<$0.1	median	p95	$<$0.1
Solver labels, as generated	1.940 ± 0.074	21.88 ± 0.34	6.8	6.273 ± 0.068	23.98 ± 0.03	0.1
Solver labels, wrapped to $(-\pi, \pi]$	1.646 ± 0.110	22.38 ± 0.19	7.1	5.700 ± 0.041	24.06 ± 0.07	0.1
Closed-form, random branch per sample	14.411 ± 0.051	29.13 ± 0.18	0.0	15.952 ± 0.198	29.83 ± 0.10	0.0
Closed-form, one branch, angle output	0.053 ± 0.001	0.13 ± 0.00	88.2	0.074 ± 0.002	0.17 ± 0.00	72.1
Closed-form, one branch, $(\cos, \sin)$ output	0.045 ± 0.000	0.11 ± 0.00	93.1	0.042 ± 0.001	0.10 ± 0.00	95.0

b) Results: Table IV and Figure 5a show a clean separation into two groups that differ by almost two orders of magnitude. Every regime trained on solver labels or on mixed branches has a median error between 1.6 and 16 LU and a 95th percentile above 21 LU, i.e. beyond the arm's reach. Every regime trained on one consistent branch has a median below 0.075 LU and a 95th percentile below 0.17 LU. Within the two groups:

- *Branch mixing is the dominant failure.* Drawing the branch at random per sample gives a median error of 14.4 LU (L1) and 16.0 LU (L2), 72–80% of the reach, and not a single target below 0.5 LU. This is the averaging pathology of direct inverse modelling [12] in its pure form. The solver's labels sit between this extreme and a consistent labelling, as their partial spatial coherence (Section V-B) suggests.
- *The loss matters only when labels are multi-valued.* With solver labels, L1 is $3.2\times$ better than L2 at the median (1.94 vs. 6.27 LU). Under L1 the network tends to pick one mode of a locally bimodal target instead of averaging between modes, which probably explains why the original MAE choice worked at all. With consistent labels, the gap between the two losses mostly disappears.
- *Wrapping helps a little.* Wrapping the solver labels to $(-\pi, \pi]$ lowers the L1 median by 15% (1.94 to 1.65 LU) but leaves the tail unchanged, because the 1.3% of unwrapped labels is a minor problem next to branch switching.
- *A continuous output head is the last step.* With consistent labels, the $(\cos, \sin)$ head improves the L2 median from 0.074 to 0.042 LU and raises the share of targets below 0.1 LU from 72% to 95%. It removes the remaining

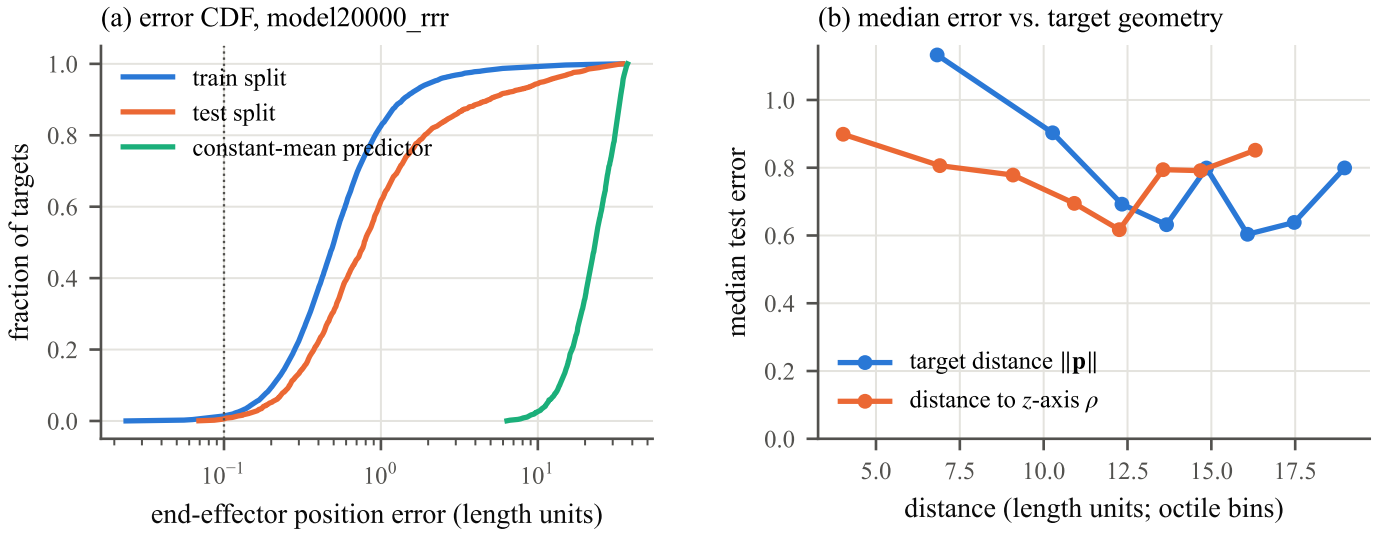


Fig. 4. Original `model20000_rrr`. **(a)** Empirical CDF of task-space error on the training and test splits, together with a constant-mean predictor. The dotted line marks 0.1 LU. **(b)** Median test error in octile bins of target distance $\|\mathbf{p}\|$ and of distance to the z-axis ρ. Error peaks near the folded-elbow singularity at the origin.

discontinuity of θ_1 and θ_2 at $\pm\pi$ [29].

Holding the inputs fixed, the best regime is therefore $43\times$ better at the median and $199\times$ better at the 95th percentile than the same network trained on the solver's labels (L1: 1.94 to 0.045 LU, and 21.9 to 0.110 LU). Our 40k-step budget does not reproduce the original model's absolute level on solver labels (median 1.94 vs. 0.76 LU, the latter after roughly 1.9×10^5 steps with a larger network). The ranking of the regimes, however, is unambiguous at this budget.

c) Scaling with exact labels: With exact labels sampled uniformly in the reachable cube and a 60k-step budget (Table V, Figure 5b), the consistent $(\cos,\sin)$ network reaches a median error of 0.039 LU from the original 7,344 inputs and 0.032 LU from 30k samples. The 99th percentile falls from 0.15 to 0.10 LU, and 99.1% of targets are within 0.1 LU. Quadrupling the data again to 120k gives no further gain (median 0.033 LU), so at this budget the error is limited by optimisation and capacity, not by data. Compared with the original `model20000_rrr` on the same 20k fresh targets, the 30k-sample model is $24\times$ better at the median (0.763 to 0.032 LU), $177\times$ better at the 95th percentile (11.9 to 0.068 LU), and $252\times$ better at the 99th (24.3 to 0.097 LU). It does this with 42% fewer parameters and without any solver in the data-generation loop.

d) Learned warm starts: A learned map can also serve as the initial guess of a numerical solver. This keeps the solver's accuracy and convergence check and uses the network only to choose where to start. Table VI compares three initial guesses on 3,000 fresh reachable targets. From $\mathbf{q}_0 = \mathbf{0}$, as in the original scripts, LM converges in 15.8 iterations on average, and only 39.5% of its solutions land on the $(+,+)$ branch; the rest are scattered over the other three. A warm start from the original network cuts the iterations to 6.2, but the branch distribution is unchanged (39.7%), because the network inherited the solver's incoherent branch choice. A warm start from the consistent network needs 4.5 iterations on average (95th percentile 5) and returns the $(+,+)$ branch for *every* target. No initialisation produced a failure on reachable targets. In this hybrid scheme the learned model contributes branch selection and speed, while the solver guarantees accuracy.

e) Latency: Table VII reports wall-clock cost in Python/NumPy. The closed-form solution is the fastest method in both single-query (12.7 µs) and batched (0.13 µs per query) use, and it is exact. The original 233k-parameter MLP is $4\times$ slower than closed form for a single query and $23\times$ slower batched. The leaner consistent network is $1.4\times$ and $11\times$ slower, respectively. The LM replica from $\mathbf{q}_0 = \mathbf{0}$ takes 358 µs on a representative target, and 89 µs from a learned warm start. Including the 18 µs network evaluation, this is a $3.3\times$ speed-up. A learned model is thus not a faster solver for an arm that has a closed form. It *is* a cheap way to accelerate and steer an iterative solver, which is the regime that matters for arms without one.

VIII. Discussion

a) What the baseline learns: The baseline network learns a coarse approximation of the RRR inverse map: its median error is 3.9% of the reach, far better than a constant predictor, and on a typical target it is within 3° of an exact solution. Its accuracy is limited by its training labels rather than its capacity. The same network trained on consistent labels is two orders of magnitude more accurate (Section VII). Each limitation of the baseline arises from a default that looks reasonable in isolation: Keras's `'accuracy'` metric, the solver's fixed initial guess, and its unwrapped output.

b) Where a learned IK model helps: For the two arms studied here, a closed-form solver is exact and faster than any network (Table VII), so the learned model has no role *as a*

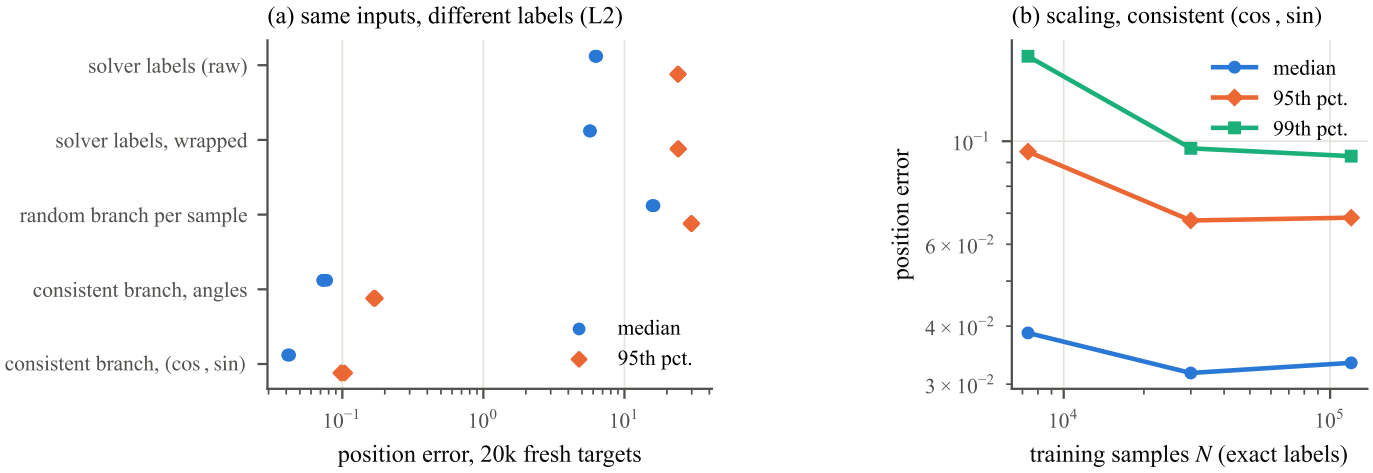


Fig. 5. **(a)** Controlled study (L2 loss): median and 95th-percentile task-space error on 20k fresh targets. The bars span three seeds. **(b)** Scaling the consistent $(\cos, \sin)$ model with exact labels drawn uniformly in the reachable cube; values are means over three seeds.

TABLE V
ORIGINAL MODEL VERSUS THE CONSISTENT $(\cos, \sin)$ MODEL TRAINED ON N EXACT LABELS (60K STEPS), EVALUATED ON THE SAME 20K FRESH REACHABLE TARGETS. ERRORS IN LU.

Model	Seeds	median	p95	p99	<0.1 LU [%]
Original `model20000_rrr` (233k params, 7.3k solver labels)	1	0.763	11.94	24.30	0.7
Consistent $(\cos, \sin)$, $N = 7{,}344$ exact labels	3	0.039 ± 0.000	0.09	0.15	95.8
Consistent $(\cos, \sin)$, $N = 30{,}000$ exact labels	3	0.032 ± 0.001	0.07	0.10	99.1
Consistent $(\cos, \sin)$, $N = 120{,}000$ exact labels	3	0.033 ± 0.000	0.07	0.09	99.3

TABLE VI
LM (`IKINE` REPLICA) ON 3,000 FRESH REACHABLE TARGETS WITH DIFFERENT INITIAL GUESSES. "TARGET BRANCH" IS THE SHARE OF SOLUTIONS ON THE $(+,+)$ BRANCH. A CONSISTENT LEARNED WARM START MAKES THE SOLVER'S OUTPUT BRANCH PREDICTABLE.

Initial guess	Fail	LM iterations			Target branch
	[%]	mean	median	p95	[%]
$\mathbf{q}_0 = \mathbf{0}$ (as in the original scripts)	0.00	15.8	14	28	39.5
$\mathbf{q}_0 =$ original `model20000_rrr`	0.00	6.2	5	11	39.7
$\mathbf{q}_0 =$ consistent $(\cos, \sin)$ network	0.00	4.5	4	5	100.0

TABLE VII
WALL-CLOCK COST PER IK QUERY IN PYTHON/NUMPY ON AN APPLE M1 PRO CPU (SINGLE QUERY: MEAN OVER 2,000 CALLS; BATCHED: 10^4 TARGETS PER CALL). ABSOLUTE VALUES DEPEND ON THE IMPLEMENTATION; RATIOS ARE THE INFORMATIVE QUANTITY.

Method	Single [µs]	Batched [µs/query]
Closed-form IK (Eq. 2)	12.7	0.130
MLP, original architecture (233k params)	50.9	3.006
MLP, consistent $(\cos, \sin)$ (134k params)	17.8	1.376
LM `ikine` replica, $\mathbf{q}_0 = \mathbf{0}$	358	–
LM `ikine` replica, learned warm start	89	–

solver. Its value lies elsewhere. (i) Arms without closed-form IK (redundant, or with offset wrists) have no exact alternative. (ii) As a warm start, a consistent learned map fixes which branch the downstream solver returns, which a fixed initial guess cannot do (Table VI). (iii) Batched, differentiable inverse maps can be embedded in planning or learning loops. In all three cases, the lessons below apply unchanged.

c) A checklist for solver-supervised IK learning:

C1. Sample where solutions exist. Sample joint space and apply FK, or reject targets outside the workspace. Do not interpret solver failures as singularities without checking reachability.

C2. Make the label a function. Either fix a branch (a canonical selection rule, or a consistent warm start) or model the multiplicity explicitly [18], [20]. Never mix branches from basin-dependent solver output.

C3. Canonicalise angles and prefer a continuous output head such as $(\cos, \sin)$ [29].

C4. Measure in task space. Report $\|\mathrm{FK}(\hat{\mathbf{q}}) - \mathbf{p}\|$ with medians *and* tails, plus wrapped joint errors. Never use a classification "accuracy" for regression.

C5. Check ranges against units. The inputs are Cartesian coordinates, not angles. Plot the training distribution against the intended workspace.

C6. Keep held-out data held out, including for qualitative figures, and state which split every plot comes from.

C7. Time it against the strongest available baseline, including closed form when it exists.

d) Limitations: MATLAB was not available to us, so the solver statistics come from a re-implementation. It matches the original data's failure rate and branch distribution, but not its angle-unwrapping frequency. All findings about the original models use the original weights and data. The controlled study uses a single small architecture and 3-DOF position IK. Redundant arms and full 6-DOF poses would strengthen the conclusions about warm starts. Latency is measured in Python and is only indicative.

IX. Conclusion

We presented and analysed a solver-supervised neural IK pipeline for two 3-DOF manipulators. Keras's default accuracy is an argmax-agreement score that does not measure IK quality. The solver's failures are workspace violations, its labels are branch-incoherent and unwrapped, and one generator sampled a Cartesian coordinate from an angle range. Measured in task space, the baseline RRR network reaches a median end-effector error of about 4% of the reach, with a heavy tail. With the inputs held fixed, consistent exact labels and a continuous output head reduce the error by $43\times$ at the median and $199\times$ at the 95th percentile. Trained on 30k exact samples, a smaller network beats the baseline model by $24\times$ at the median and $177\times$ at the 95th percentile. Used as a warm start, a consistent learned map makes a numerical solver converge $3.5\times$ faster and always return the intended branch. The broader lesson is that, in learned IK, the label-generation procedure and the evaluation metric need as much scrutiny as the network itself.

Code and Data Availability

The original project artifacts and the analysis code (`paper/code`: `audit.py`, `improved.py`, `figures.py`, `make_tables.py`) regenerate every number, table, and figure in this report.

Acknowledgment

The pipeline analysed here was built for Project 2 of MAE 547 (Modeling and Control of Robots) at Arizona State University, Fall 2021, by the author together with Rumitkumar Patel and Priyank Patel. The author thanks them for their work on the original artifacts. The analysis, re-evaluation, controlled study, and this report are the author's own subsequent work. The 2026 re-analysis was carried out with the assistance of an AI coding assistant; all results were produced by the released scripts, and the author takes full responsibility for the content.

References

[1] J. Denavit and R. S. Hartenberg, "A kinematic notation for lower-pair mechanisms based on matrices," *Journal of Applied Mechanics*, vol. 22, pp. 215–221, 1955.

[2] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2009.

[3] D. L. Pieper, "The kinematics of manipulators under computer control," Ph.D. dissertation, Stanford University, 1968.

[4] M. Raghavan and B. Roth, "Inverse kinematics of the general 6R manipulator and related linkages," *Journal of Mechanical Design*, vol. 115, no. 3, pp. 502–508, 1993.

[5] R. Diankov, "Automated construction of robotic manipulation programs," Ph.D. dissertation, Carnegie Mellon University, 2010.

[6] C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[7] Y. Nakamura and H. Hanafusa, "Inverse kinematic solutions with singularity robustness for robot manipulator control," *Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[8] S. R. Buss, "Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods," University of California, San Diego, Tech. Rep., 2004.

[9] T. Sugihara, "Solvability-unconcerned inverse kinematics by the Levenberg–Marquardt method," *IEEE Transactions on Robotics*, vol. 27, no. 5, pp. 984–991, 2011.

[10] P. Beeson and B. Ames, "TRAC-IK: An open-source library for improved solving of generic inverse kinematics," in *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 2015.

[11] A. Guez and Z. Ahmad, "Solution to the inverse kinematics problem in robotics by neural networks," in *IEEE International Conference on Neural Networks*, 1988, pp. 617–624.

[12] M. I. Jordan and D. E. Rumelhart, "Forward models: Supervised learning with a distal teacher," *Cognitive Science*, vol. 16, no. 3, pp. 307–354, 1992.

[13] A. D'Souza, S. Vijayakumar, and S. Schaal, "Learning inverse kinematics," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2001, pp. 298–303.

[14] E. Oyama, A. Agah, K. F. MacDorman, T. Maeda, and S. Tachi, "A modular neural network architecture for inverse kinematics model learning," *Neurocomputing*, vol. 38–40, pp. 797–805, 2001.

[15] B. Bócsi, D. Nguyen-Tuong, L. Csató, B. Schölkopf, and J. Peters, "Learning inverse kinematics with structured prediction," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2011, pp. 698–703.

[16] A. Csiszar, J. Eilers, and A. Verl, "On solving the inverse kinematics problem using neural networks," in *International Conference on Mechatronics and Machine Vision in Practice (M2VIP)*, 2017, pp. 1–6.

[17] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.

[18] C. M. Bishop, "Mixture density networks," Aston University, Tech. Rep. NCRG/94/004, 1994.

[19] L. Ardizzone, J. Kruse, S. Wirkert, D. Rahner, E. W. Pellegrini, R. S. Klessen, L. Maier-Hein, C. Rother, and U. Köthe, "Analyzing inverse problems with invertible neural networks," in *International Conference on Learning Representations (ICLR)*, 2019.

[20] B. Ames, J. Morgan, and G. Konidaris, "IKFlow: Generating diverse inverse kinematics solutions," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7177–7184, 2022.

[21] P. I. Corke, "A robotics toolbox for MATLAB," *IEEE Robotics & Automation Magazine*, vol. 3, no. 1, pp. 24–32, 1996.

[22] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed., ser. Springer Tracts in Advanced Robotics. Springer, 2017, vol. 118.

[23] F. Chollet *et al.*, "Keras," https://keras.io, 2015.

[24] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

[25] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

Algorithm 1 Position-only LM IK (replica of RTB `SerialLink.ikine` as used)

Require: target $\mathbf{p}^\star$, initial $\mathbf{q}_0$; $\lambda \leftarrow 0.1$, $\varepsilon = 10^{-10}$, $I_{\max} = 1000$, $R_{\max} = 100$

$\mathbf{q} \leftarrow \mathbf{q}_0$, $\mathbf{e} \leftarrow \mathbf{p}^\star - \mathrm{FK}(\mathbf{q})$, $k \leftarrow 0$, $r \leftarrow 0$
loop
  $k \leftarrow k + 1$; **if** $k > I_{\max}$ **return** FAIL
  if $\|\mathbf{e}\| < \varepsilon$ **return** $\mathbf{q}$
  $J \leftarrow \partial \mathrm{FK}/\partial \mathbf{q}\,(\mathbf{q})$; $\Delta\mathbf{q} \leftarrow (J^\top J + \lambda I)^{-1} J^\top \mathbf{e}$
  if $\|\mathbf{p}^\star - \mathrm{FK}(\mathbf{q} + \Delta\mathbf{q})\| < \|\mathbf{e}\|$ **then**
    $\mathbf{q} \leftarrow \mathbf{q} + \Delta\mathbf{q}$, $\mathbf{e} \leftarrow \mathbf{p}^\star - \mathrm{FK}(\mathbf{q})$, $\lambda \leftarrow \lambda/2$, $r \leftarrow 0$
  else
    $\lambda \leftarrow 2\lambda$, $r \leftarrow r + 1$; **if** $r > R_{\max}$ **return** FAIL
  end if
end loop

[26] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[27] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quarterly of Applied Mathematics*, vol. 2, no. 2, pp. 164–168, 1944.

[28] D. W. Marquardt, "An algorithm for least-squares estimation of non-linear parameters," *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963.

[29] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5745–5753.

[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[31] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

APPENDIX A
LM SOLVER RE-IMPLEMENTATION

With a position-only mask, the toolbox's end-effector-frame error and Jacobian give the same step as the world-frame form above, since the rotation cancels in $J^\top W \mathbf{e}$. Rejected steps count towards the iteration limit, as in the toolbox loop.

APPENDIX B
REPRODUCTION

```
pip install h5py numpy pandas scipy \
            scikit-learn matplotlib torch
cd paper/code
python audit.py      # Part I  (~30 s)
python improved.py   # Part II (~45 min CPU)
python figures.py; python make_tables.py
cd .. && tectonic main.tex
```

No TensorFlow installation is needed: Keras weights are read directly from HDF5 and evaluated in NumPy.